

Capstone - Modulo 06

Servizio di stima dei tempi di consegna

LogiFast Solutions - deployment di un modello predittivo via API REST

AI Service Deployment

Simone La Porta

Master AI Solutions Architect - ProfessionAI

Indice

1. Contesto e valore.....	3
2. Dati.....	3
3. Analisi esplorativa e validazione.....	3
4. Scelta delle metriche.....	4
5. Architettura della soluzione.....	4
6. Elementi MLOps.....	4
7. Piano di rollout.....	4
8. Limiti e sviluppi.....	5
Appendice A - Schema dei dati.....	6
Input (record di ordine).....	6
Output (predizione).....	6
Appendice B - Specifica API REST.....	8
POST /predict.....	8
POST /predict/batch.....	8
GET /health.....	9
GET /model.....	9
Formato degli errori.....	9
Codici di stato.....	9
Appendice C - Disegno MLOps.....	10
1. Versionamento.....	10
2. Automazione (pipeline concettuale).....	10
3. Monitoraggio.....	11
4. Governance.....	12
Appendice D - Avvio rapido e repository.....	13

Cliente: LogiFast Solutions (fornitore logistico urbano e interurbano). **Modulo 06 - AI Service Deployment.**
Autore: Simone La Porta.

1. Contesto e valore

LogiFast vuole stimare il tempo di consegna (time-to-delivery) di ogni ordine al momento della presa in carico, per informare clienti e operatori, pianificare le rotte e ridurre reclami e reinvii. La soluzione espone un modello predittivo via API REST in Flask, consumabile dal gestionale e dal front-end e-commerce.

Il valore non sta solo nella stima puntuale ma nell'**affidabilità del servizio nel tempo**: versionamento e monitoraggio permettono di accorgersi quando il modello smette di essere accurato e di intervenire, invece di scoprirlo dai reclami.

2. Dati

Input noti alla presa in carico: origine, destinazione, data/ora di ritiro, peso, tipo di servizio. Output: stima in ore, con score di affidabilità e banda indicativa. Lo schema completo e i vocabolari (20 città, servizi Express/Premium) sono in *Appendice A*.

Un primo risultato dell'esplorazione è una **discrepanza tra traccia e artefatto**: la traccia elenca cinque campi di input, ma il modello fornito ne usa quattro - `pickup_datetime` non entra nella predizione. L'API lo accetta comunque (contratto stabile in vista del riaddestramento) e lo dichiara come non usato.

3. Analisi esplorativa e validazione

Il dettaglio è in [notebooks/exploration_validation.ipynb](#). L'artefatto è una Pipeline scikit-learn: `OneHotEncoder` sulle categoriche (città, servizio) + `passthrough` sul peso, seguito da una `LinearRegression`.

Due letture, una intrinseca e una su dati.

- **Intrinseca** (solo coefficienti e risposte del modello): il modello è poco sensibile agli input. Il peso da 1 a 50 kg sposta la stima di ~2 ore, Express e Premium di mezz'ora, le destinazioni restano in una banda stretta attorno alle 47 ore. Una regressione lineare su one-hot di città cattura un effetto additivo per città, **non la distanza** tra origine e destinazione.
- **Su dati sintetici** (in assenza di uno storico etichettato, dataset generato da un processo documentato che incorpora la distanza geografica reale tra le città): MAE 20.2 ore, RMSE 22.5 ore, **R2 = -4.82**. Un R2 negativo significa che il modello fa peggio di una semplice media: predice ~costante mentre il tempo reale varia molto con la distanza.

La conclusione non è "il modello è inutile": la traccia chiede di deployarlo, ed è quello che facciamo. La conclusione è che il suo limite è noto, misurato e sistematico (errore sui tragitti lunghi), quindi **gestibile** dal monitoraggio invece che subito.

4. Scelta delle metriche

MAE e RMSE perché il problema è una regressione e l'errore va letto nell'unità di business (ore): MAE per l'errore tipico, RMSE perché penalizza di più gli errori grossi, che qui sono concentrati sui tragitti lunghi. R2 per posizionare il modello rispetto a una baseline (la media): è la metrica che ha reso evidente il problema. In produzione la metrica chiave diventa l'**errore per segmento** (tratta, peso, servizio): la media aggregata nasconderebbe proprio il punto debole.

5. Architettura della soluzione

```
Client (e-commerce / operatori)
  | JSON
  v
Flask app (app.py)                <- routing, validazione payload, logging
  |
  v
DeliveryModelService              <- validazione di dominio, affidabilità, metadati
  |
  v
Pipeline sklearn (delivery.pkl) <- OHE(città, servizio) + peso -> LinearRegression
  |
  v
Risposta JSON                     <- stima (ore) + intervallo + reliability + warning
```

Due piani separati di proposito: trasporto HTTP (`app.py`) e contratto del modello (`model_service.py`). La logica di predizione è testabile senza server e il modello non dipende da Flask. Endpoint: POST `/predict`, POST `/predict/batch`, GET `/health`, GET `/model`. Dettaglio in *Appendice B*.

Scelte difese altrove e qui sintetizzate: `pickup_datetime` accettato ma segnalato come non usato; città fuori vocabolario gestite con warning e affidabilità ridotta invece che con un errore; banda dell'intervallo pari all'RMSE sintetico (~22.5 ore), ampia perché onesta sul modello debole; sicurezza del pickle (verifica statica + hash SHA-256).

6. Elementi MLOps

Versionamento (dataset, modello, model card), pipeline concettuale di automazione, monitoraggio (logging già attivo, metriche runtime e di qualità per segmento, drift, alerting) e governance (rollback su artefatti versionati, trigger di riaddestramento) sono progettati in *Appendice C*. Il prossimo ciclo di training ha un candidato ovvio: aggiungere distanza e `pickup_datetime`, le informazioni che il modello attuale ignora.

7. Piano di rollout

Promozione a stadi, ciascuno con criterio di avanzamento e di rollback.

1. **Staging**: la suite `tests/test_api.py` passa, l'immagine Docker si avvia, `/health` e `/model` rispondono.
2. **Shadow**: il servizio gira in parallelo alla procedura attuale, le predizioni sono loggate ma non mostrate. Si raccoglie ground truth e si misura l'errore per segmento.
3. **Canary**: una quota ridotta di traffico reale (es. 5-10%). Avanzamento se latenza e errore restano nei limiti; rollback se un alert critico scatta.
4. **Full**: rollout completo, con monitoraggio e trigger di riaddestramento attivi.

8. Limiti e sviluppi

- Il modello ignora distanza e data/ora: è il limite dominante, già misurato. Sviluppo prioritario: riaddestrare includendole.
- L'unità (ore) è un'assunzione sul range osservato, da confermare con la documentazione di training.
- `reliability_score` e intervallo sono proxy dichiarati, non statistiche rigorose: con dati reali si possono sostituire con intervalli di predizione veri.
- La validazione è su dati sintetici: serve uno storico etichettato per metriche di produzione affidabili.

Appendice A - Schema dei dati

Input (record di ordine)

Campi accettati in `POST /predict` e in ogni elemento di `POST /predict/batch`.

Campo	Tipo	Obbligatorio	Vincoli	Usato dal modello
<code>pickup_location</code>	stringa	si	città di origine	si (one-hot)
<code>delivery_location</code>	stringa	si	città di destinazione	si (one-hot)
<code>pickup_datetime</code>	stringa	si	ISO 8601 consigliato	no (vedi nota)
<code>weight</code>	numero	si	> 0, in kg	si (passthrough)
<code>service_type</code>	stringa	si	Express o Premium	si (one-hot)

Nota su ``pickup_datetime``. Il modello fornito è addestrato su quattro feature e non usa la data/ora di ritiro. Il campo resta nel contratto per coerenza con la specifica e per non rompere i client quando il modello verrà riaddestrato includendolo. La validazione è lasca: si tenta il parse ISO 8601 e si avvisa, senza bloccare. L'endpoint `/model` lo elenca in `unused_fields`.

Vocabolari chiusi

- ``service_type``: `Express`, `Premium`. Un valore fuori da questo insieme produce un errore 400: è un insieme di business chiuso, non ha senso degradare silenziosamente.
- ``pickup_location`` / ``delivery_location``: 20 città note, apprese dall'artefatto e esposte da `/model`: Ancona, Bari, Bologna, Cagliari, Catania, Firenze, Genova, Lecce, Milano, Napoli, Palermo, Perugia, Pescara, Reggio Calabria, Roma, Salerno, Sassari, Torino, Trapani, Verona. Una città fuori vocabolario **non** genera errore (l'`OneHotEncoder` usa `handle_unknown='ignore'`, la codifica a zeri) ma abbassa il `reliability_score` e aggiunge un warning. La scelta è deliberata: una stima silenziosamente degradata è peggio di una segnalata.

Output (predizione)

Campo	Tipo	Descrizione
<code>predicted_delivery_time_hours</code>	numero	stima del tempo di consegna, in ore
<code>confidence_interval_hours</code>	[numero, numero]	banda indicativa (~RMSE sintetico, ~22.5 h), non statistica
<code>reliability_score</code>	numero in [0, 1]	euristica di in-distribution, non una probabilità
<code>unit</code>	stringa	ore
<code>model_version</code>	stringa	versione dell'artefatto deployato
<code>warnings</code>	lista di stringhe	anomalie non bloccanti

Campo	Tipo	Descrizione
input	oggetto	record validato (solo in /predict)

Unità di misura. Il modello restituisce un numero senza unità. Sul range osservato (circa 39-55, media ~46) l'interpretazione coerente con consegne interurbane è in ore (~2 giorni medi), non minuti. Assunzione dichiarata, da confermare con la documentazione di training del modello.

`reliability\_score`. Parte da 1.0 e viene moltiplicato per 0.6 per ogni città fuori vocabolario e per 0.9 se il peso supera i 100 kg. Non è una probabilità: una `LinearRegression` non espone `predict_proba`. È un proxy di quanto l'input cade nella distribuzione su cui il modello è affidabile.

Appendice B - Specifica API REST

Base URL (sviluppo): `http://localhost:8080`. Tutte le richieste e risposte sono in `application/json`. Lo schema dei campi è in *Appendice A*.

POST /predict

Predizione su un singolo ordine.

Richiesta

```
{
  "pickup_location": "Milano",
  "delivery_location": "Roma",
  "pickup_datetime": "2026-07-01T09:30:00",
  "weight": 5.0,
  "service_type": "Express"
}
```

Risposta 200

```
{
  "predicted_delivery_time_hours": 43.07,
  "confidence_interval_hours": [20.57, 65.57],
  "reliability_score": 1.0,
  "unit": "ore",
  "model_version": "1.0.0",
  "warnings": [],
  "input": {
    "pickup_location": "Milano", "delivery_location": "Roma",
    "pickup_datetime": "2026-07-01T09:30:00", "weight": 5.0,
    "service_type": "Express"
  }
}
```

Con input fuori distribuzione la risposta resta 200 ma `reliability_score` scende e `warnings` si popola (es. città ignota, `pickup_datetime` non parsabile).

POST /predict/batch

Predizione su una lista di ordini. Un record malformato non fa cadere il batch: viene isolato come errore, gli altri vengono predetti.

Richiesta: lista di record con lo stesso schema di `/predict`.

Risposta 200

```
{
  "count": 2,
  "results": [
    {
      "index": 0, "status": "ok", "predicted_delivery_time_hours": 55.19,
      "confidence_interval_hours": [32.69, 77.69], "reliability_score": 1.0,
      "unit": "ore", "model_version": "1.0.0", "warnings": []
    },
    {
      "index": 1, "status": "error", "errors": ["weight deve essere > 0"]
    }
  ]
}
```


GET /health

Stato operativo del servizio (readiness). `status` è `OK` con `200` se il modello è caricato, `ERROR` con `503` se l'artefatto non è disponibile: dietro un orchestratore serve a non ricevere traffico finché il servizio non è pronto.

```
{ "status": "OK", "timestamp": "2026-07-01T07:49:00.152448+00:00" }
```

GET /model

Versione e contratto del modello: utile per i client e per il versionamento.

```
{
  "model_version": "1.0.0",
  "model_type": "LinearRegression",
  "framework": "scikit-learn 1.6.1",
  "pinned_sklearn": "1.6.1",
  "features_used": ["pickup_location", "delivery_location", "weight", "service_type"],
  "request_fields": ["pickup_location", "delivery_location", "pickup_datetime", "weight", "service_type"],
  "unused_fields": ["pickup_datetime"],
  "output_unit": "ore",
  "known_locations": ["Ancona", "... 20 città ..."],
  "known_service_types": ["Express", "Premium"],
  "artifact_sha256": "cb29574f...",
  "loaded_at": "2026-06-30T21:49:00+00:00"
}
```

Formato degli errori

Gli errori bloccanti hanno sempre la stessa forma, con la lista puntuale dei problemi:

```
{ "status": "error", "errors": ["campi mancanti: weight"] }
```

Codici di stato

Codice	Quando
200	richiesta valida (anche con warning per input degradato)
400	payload assente/non-JSON, campi mancanti, <code>service_type</code> non valido, body batch non lista
404	endpoint inesistente
500	errore interno non previsto (loggato lato server)
503	modello non disponibile (<code>/health</code> risponde <code>ERROR</code> , <code>/predict</code> e <code>/predict/batch</code> rifiutano)

Appendice C - Disegno MLOps

Come da traccia, questi elementi sono **progettati e documentati**, non implementati come ecosistema reale. Dove esiste già un gancio nel codice (logging, hash dell'artefatto, endpoint `/model`, validazione con warning) lo richiamo: il disegno non parte da zero, estende quello che il servizio espone.

Il filo conduttore è una scoperta della validazione (vedi `notebooks/`): il modello fornito **ignora la distanza** tra origine e destinazione e predice un valore quasi costante (R^2 negativo, -4.82, contro una realtà distanza-aware). Un buon disegno MLOps deve rendere questo degrado **visibile e azionabile**, non nascondere.

1. Versionamento

Dataset

Ogni dataset di training è uno snapshot immutabile, identificato da `dataset_id` (es. `delivery-2026-06`) e da un hash del contenuto. Si versionano: la finestra temporale, lo schema, le 20 città in vocabolario, le statistiche di base (distribuzioni di peso, tratte, servizio). In assenza di un data lake, anche un semplice manifest versionato (CSV + `manifest.json` con hash) è sufficiente a garantire riproducibilità.

Modello e metadati

L'artefatto è versionato con SemVer (`MODEL_VERSION`, oggi `1.0.0`) e accompagnato da una **model card** con: tipo (`LinearRegression`), feature usate, `dataset_id` di training, metriche di validazione, data, commit Git, e l'hash SHA-256 dell'artefatto (già calcolato al load in `model_service.py`). L'endpoint `/model` espone questi metadati a runtime.

Compatibilità del contratto

Il contratto API (`request_fields`) è più ampio delle feature del modello: include `pickup_datetime`, oggi inutilizzato. Questo è deliberato e va versionato: quando il modello verrà riaddestrato includendo la data/ora, i client esistenti non cambieranno. Una modifica che rimuovesse o rinominasse un campo richiesto sarebbe invece breaking e imporrebbe un bump di major.

2. Automazione (pipeline concettuale)

```
commit / nuovo dataset
|
v
[1] Qualità dati      -> schema, range, vocabolario città, % nulli
|
v
[2] Training          -> fit Pipeline, salva artefatto + model card
|
v
[3] Validazione       -> metriche su test, gate vs baseline e vs versione in prod
|
v
[4] Build & deploy     -> immagine Docker, push su registry, deploy in staging
|
v
[5] Promozione        -> shadow -> canary -> full (vedi rollout nel report)
```

[1] Qualità dei dati

Controlli automatici prima del training: schema atteso, `weight > 0`, `service_type` nel vocabolario, città note, percentuale di valori mancanti sotto soglia. Un fallimento blocca la pipeline: meglio non addestrare che addestrare su dati sporchi.

[2] Training

Riproducibile: seed fisso, dipendenze pinnate (`requirements.txt`, `scikit-learn==1.6.1`), `dataset_id` registrato. L'output è l'artefatto più la model card.

[3] Test e validazione pre-deploy

Due cancelli. Il primo, **test di regressione del modello**: le metriche sul set di test non devono peggiorare oltre una soglia rispetto alla versione in produzione (no regressioni silenziose). Il secondo, **test del servizio**: la suite `tests/test_api.py` deve passare sulla nuova immagine (endpoint, validazione, batch). Solo con entrambi verdi si procede.

[4] Build e deploy

Immagine Docker costruita dalla pipeline, taggata con `MODEL_VERSION` e commit, spinta su un registry. Il deploy in staging precede sempre la produzione.

3. Monitoraggio

Logging di richieste e predizioni

Già attivo: `app.py` logga per ogni richiesta la tratta, il servizio, la stima e il `reliability_score`. In produzione questi log alimentano un archivio strutturato (una riga per predizione: input, output, versione modello, timestamp) che è la base di tutto il resto.

Metriche runtime

Latenza (p50/p95/p99), throughput, tasso di errori 4xx/5xx, quota di richieste con `reliability_score` basso. Sono le metriche di servizio, indipendenti dalla qualità del modello.

Metriche di qualità

Richiedono il **ground truth differito**: il tempo di consegna effettivo, noto solo dopo la consegna. Quando arriva, si accoppia alla predizione loggata e si calcola l'errore (MAE, RMSE) **per segmento**: per tratta, fascia di peso, servizio. Il monitoraggio per segmento non è un dettaglio: la validazione ha mostrato che l'errore di questo modello è concentrato sui tragitti lunghi, e una media aggregata lo nasconderebbe.

Drift detection

Due piani. **Data drift**: la distribuzione degli input cambia rispetto al training (nuove città, pesi fuori range, mix di servizi diverso) - intercettabile già dai warning e dal `reliability_score`, aggregati nel tempo. **Concept drift**: la relazione input-tempo cambia (nuovi corrieri, congestione) - visibile come deriva dell'errore per segmento.

Alerting

Soglie su: errore per segmento oltre una banda (es. $RMSE > 1.5 \times$ baseline su una tratta), quota di predizioni a bassa affidabilità in crescita, comparsa stabile di città fuori vocabolario, p95 di latenza oltre l'SLA. L'alert è azionabile: indica il segmento, non solo "il modello peggiora".

4. Governance

Rollback

Gli artefatti sono versionati e immutabili: il rollback è ridistribuire l'immagine con la `MODEL_VERSION` precedente. L'endpoint `/model` e l'hash SHA-256 permettono di verificare quale versione è davvero in esecuzione. Criterio: si effettua rollback se la nuova versione viola un gate di qualità in canary o se un alert critico scatta entro la finestra di osservazione.

Trigger di riaddestramento

- **Degrado:** errore per segmento oltre soglia su una finestra mobile.
- **Drift dei dati:** spostamento stabile della distribuzione degli input.
- **Nuovi dati:** disponibilità di uno storico etichettato sufficiente (es. trimestrale).
- **Gap noto:** il candidato naturale del prossimo ciclo è includere la distanza tra città e la `pickup_datetime`, le due informazioni che il modello attuale ignora.

Tracciabilità e audit

Ogni predizione è ricostruibile: input loggato, `model_version`, hash dell'artefatto, timestamp. Questo chiude il cerchio tra dato, modello e decisione, ed è il prerequisito per qualunque analisi a posteriori di un reclamo o di un errore sistematico.

Appendice D - Avvio rapido e repository

Il servizio, il modello e i test sono nel repository pubblico del progetto. Richiede Python 3.12+. Avvio in sviluppo e in produzione:

```
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt          # scikit-learn==1.6.1 pinnato all'artefatto
python app.py                          # sviluppo, porta 8080
gunicorn -w 2 -b 0.0.0.0:8080 "app:create_app()" # produzione
```

Esempio di chiamata:

```
curl -X POST http://localhost:8080/predict \
-H "Content-Type: application/json" \
-d '{"pickup_location":"Milano","delivery_location":"Roma",
    "pickup_datetime":"2026-07-01T09:30:00","weight":5.0,
    "service_type":"Express"}'
```

Repository del progetto (pubblico):

https://github.com/Shurtug4l/AI_solutions_architect_course/tree/master/06_AI_services_deployment/_PRJ_logifast_delivery_time_service